Regression Enables Adaptive Inference for Autonomous Driving Detectors

Hyungseop Lee, Jiho Lee, Sunwoo Lee, and Woochul Kang, *Member, IEEE*

***Abstract*—Dynamic-depth neural networks can trade accuracy for computation by selecting, per input, between a shallow (base) and a deep (super) execution path, but doing so requires a router that decides when the extra depth is worth its cost. Existing efficiency-aware routers are typically trained with a weighted sum of an accuracy term and a computation (FLOPs) term, frequently incorporating an additional uniformity or entropy term. This couples the operating point to training: the trade-off weight bakes a single accuracy–efficiency point into the model, while collapse onto one path must be prevented with auxiliary terms, and serving a new compute budget demands retraining. We reformulate depth routing as *advantage regression*. Instead of learning a discrete policy, a lightweight head predicts, for each image, how much detection loss the deep path would save over the shallow path. Consequently, routing reduces to simply comparing this predicted advantage against a threshold at deployment time. A single trained router therefore produces an entire continuous accuracy–efficiency Pareto curve through a simple threshold sweep, with no trade-off hyperparameter and no collapse-prevention term. On KITTI (trained on detection images, evaluated on tracking videos), the proposed router dominates random, luminance, edge-density, and confidence-based baselines, and the result transfers to BDD100K. The router adds negligible overhead, a few thousandths of a percent of the detector computation, and yields measurable on-device latency, energy, and frame-rate improvements.**

***Index Terms*—Autonomous Driving, Adaptive inference, Conditional Computation, Dynamic Neural Networks, Efficient Deep Learning, Object Detection, Accuracy–Efficiency Trade-off,**

I. Introduction

OBJECT detectors deployed in autonomous driving systems operate under strict compute, latency, and energy constraints. These constraints arise from the need for real-time perception in safety-critical driving scenarios, motivating growing interest in dynamic inference, which allows models to adjust their computational cost on the fly depending on the input [1]–[3].

Recent advances in dynamic detection have enabled a single model to operate under multiple computation regimes. In particular, AnyDepth-YOLO [1] introduces a single detector that supports multiple executable paths with different computation depths, including a *super* path (high accuracy, high computation) and a *base* path (low computation with a modest accuracy drop). This design enables input-dependent accuracy–efficiency trade-offs within a unified detector. While this provides the architectural and training foundations for dynamic execution, it does not address how to dynamically select the optimal execution depth for a given input, nor how to adjust these selections when the system's available compute or latency budget fluctuates. A fundamental question therefore remains unresolved: *how should one decide which execution depth to use for a given input under varying computation budgets?*

A natural intuition is that routing should follow semantic scene difficulty: visually complex scenes may require the super path, while simpler scenes are sufficient for the base path. However, Figure 1 shows that this intuition does not reliably hold in practice. Even in visually challenging scenes, the performance gain from deeper execution can be negligible. Table I further demonstrates that the benefit of deeper execution remains consistent across broad semantic attributes such as time of day, weather, and scene type. For example, the SUPER–BASE performance gap is nearly identical between daytime and nighttime images ($+1.22$ vs. $+1.23$ AP), and remains similar across highway and city-street scenes ($+1.15$ vs. $+1.13$ AP). These observations suggest that semantic scene attributes are insufficient to characterize when deeper computation is beneficial, motivating a learned routing mechanism that estimates the expected benefit of additional depth on a per-image basis.

While the need for a learned per-image routing mechanism is clear, establishing an effective policy presents significant challenges. Conventional approaches to dynamic inference—primarily developed within the context of image classification—typically formulate routing as a discrete decision-making problem. These methods rely on efficiency-aware objectives that balance accuracy and computational cost, often implemented via Gumbel-Softmax relaxation [4], [5] or policy gradients [6]. However, such approaches inherently embed a single accuracy–efficiency trade-off into the learned policy, effectively restricting it to a fixed operating regime determined by a trade-off coefficient, and making adaptation to different system budgets difficult without retraining. Moreover, the discrete optimization process is often unstable and prone to issues such as path collapse, requiring additional heuristics such as entropy regularization and carefully tuned balancing strategies. In contrast, routing path in object detection remains largely underexplored, and our goal is fundamentally different from these classification-oriented formulations. Rather than assigning each input to a single optimal execution path, we seek a routing mechanism that can adapt across varying latency and compute budgets, producing a spectrum of input-dependent routing behaviors rather than collapsing to a single solution.

To address this, we reformulate depth routing in object detection as an *advantage regression* problem. With the detector weights frozen, the performance gain of executing the super

The authors are with the Department of Embedded Systems Engineering, Incheon National University, Incheon, Republic of Korea (e-mail: wchkang@inu.ac.kr).

Corresponding author: Woochul Kang.

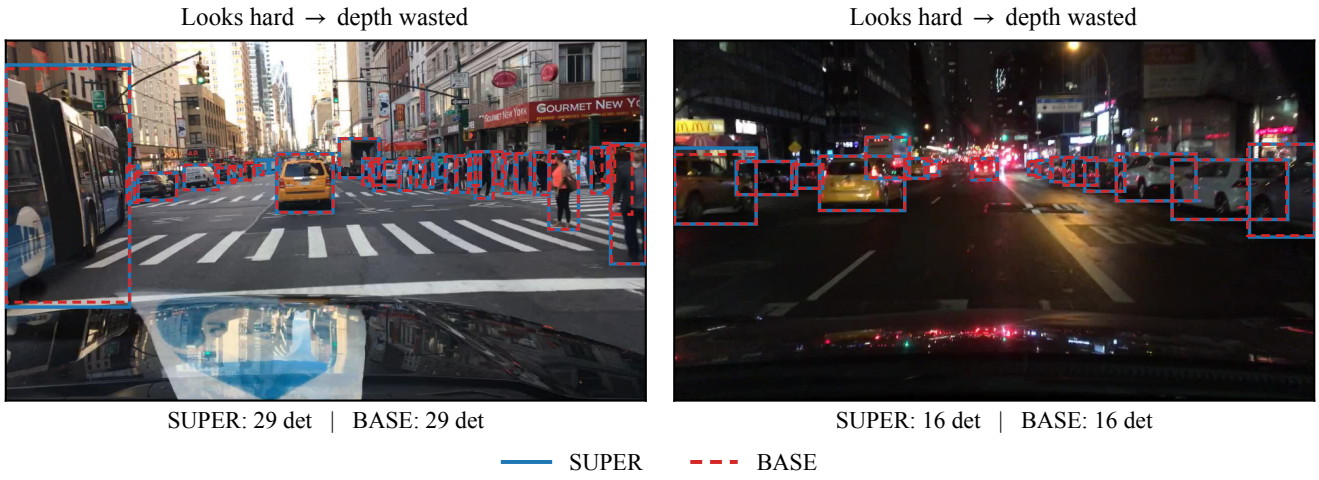


Fig. 1. The per-image benefit of the deep path is not predictable from how hard a scene looks to a human. **Left:** a cluttered daytime intersection; **Right:** a busy night scene—both look clearly difficult, yet the base path (dashed red) detects exactly as many objects as the super path (solid blue), with the boxes nearly coinciding (29 vs. 29 and 16 vs. 16). In these visually hard scenes the extra depth is wasted—so semantic appearance (clutter, crowding, lighting) is a poor predictor of where the deep path actually pays off, motivating a learned per-image estimate of its benefit. Detections are from AnyDepth-YOLOv12s [1] on BDD100K.

path over the base path becomes a fixed, measurable quantity. This allows us to move beyond discrete policy learning; instead of treating routing as a stochastic decision-making process, we train a lightweight predictor to directly regress this per-image advantage from intermediate detector features. By transforming the problem into a standard supervised regression task, we eliminate the need for policy sampling, complex exploration strategies, and auxiliary collapse-prevention objectives.

At deployment, routing reduces to a simple threshold test on the predicted advantage. The threshold directly controls the accuracy–efficiency trade-off: lower values favor the *super* path, whereas higher values route more inputs to the *base* path. As a result, training becomes fully decoupled from deployment-time budget selection. A single trained router can realize a continuous spectrum of operating points through threshold sweeping alone, yielding an accuracy–efficiency trade-off curve without retraining or objective reweighting. Furthermore, because the operating point is governed by a single scalar threshold, the same router naturally supports online adaptation to time-varying latency budgets through lightweight feedback control.

Evaluations across three representative autonomous driving datasets (KITTI [7], BDD100K [8], and Waymo [9]) demonstrate that the proposed router effectively maintains the heavy baseline's performance while significantly trimming redundant computations. Rather than compromising accuracy for speed, the router reliably achieves near-lossless detection: it reduces per-frame latency and energy consumption by approximately $10-12\%$ on KITTI and BDD100K, and over $15-16\%$ on the high-resolution Waymo benchmark, all while maintaining AP within a strictly marginal 0.1 point of the SUPER-only ceiling. Moreover, the proposed formulation offers benefits beyond a single fixed operating point. By combining the router with a simple proportional–integral controller, the system can dynamically adapt the threshold on a frame-by-frame basis to seamlessly track time-varying latency targets.

TABLE I
CONDITION-STRATIFIED AP@[.50:.95] ON THE BDD100K VALIDATION SET FOR THE FROZEN ANYDEPTH-YOLOV12S. THE BENEFIT OF DEEPER EXECUTION VARIES ACROSS INPUTS BUT IS NOT WELL EXPLAINED BY BROAD SEMANTIC SCENE CATEGORIES.

Condition	N	SUPER	BASE	Gap
All	10,000	33.23	32.05	+1.18
Time of day				
Daytime	5,258	34.76	33.54	+1.22
Night	3,929	29.75	28.52	+1.23
Dawn/Dusk	778	33.28	32.22	+1.06
Weather				
Clear	5,346	32.24	31.21	+1.03
Overcast	1,239	33.75	32.80	+0.96
Rainy	738	33.03	31.87	+1.16
Snowy	769	31.77	31.10	+0.66
Scene type				
Highway	2,499	29.03	27.88	+1.15
City street	6,112	33.54	32.42	+1.13
Residential	1,253	38.00	36.66	+1.35

II. RELATED WORK

A. Dynamic Inference in Object Detection

Adaptive inference dynamically allocates computation based on the input rather than applying a fixed static cost to every sample. While extensively explored in image classification—via early-exit networks [10]–[13], spatially adaptive computation [14], and slimmable or skippable architectures [6], [15]–[19]—this paradigm has only recently been extended to the more complex domain of object detection. Recent efforts include early-exit detectors [3], [20], multi-path subnetworks [21], deadline-aware LiDAR detectors [22], and cascaded systems like DynamicDet [2].

We build upon AnyDepth-YOLO [1], which exposes multiple execution depths from a single set of weights by selectively

bypassing refinement stages. Unlike prior works that focus heavily on the architectural design of these conditional models, we focus on the orthogonal and critical challenge: *how to optimally govern the routing decision at inference time*.

B. Learned Routing for Dynamic Inference

Many dynamic-inference methods—including SkipNet [18], BlockDrop [6], AR-Net [5], and cost-aware NAS [23]—formulate routing as a discrete policy optimized via a weighted sum of accuracy and computation. This policy-centric template carries two fundamental structural flaws. First, a fixed computation weight ties the learned router to a single operating point, meaning adaptation to a new budget inevitably requires retraining. Second, discrete routing policies inherently suffer from mode collapse, heavily favoring a single path and necessitating fragile workarounds like auxiliary load-balancing losses [24]–[26]. Even DynamicDet [2], the closest work to ours, fails to fully escape this architectural trap. While it circumvents the first flaw by employing a settable threshold to expose a continuous trade-off without retraining, it still optimizes a routing policy against difficulty-weighted detection losses. Consequently, the objective predictably collapses onto the more accurate branch, forcing the introduction of a dedicated adaptive-offset mechanism to artificially balance the routes. Furthermore, by cascading two independent models, it inherently doubles the parameter storage on the deeper route.

In contrast, our work formulates routing as a supervised advantage-regression problem. Rather than optimizing a routing policy under an efficiency-aware objective, the router directly predicts the expected benefit of deeper execution from frozen detector features. This formulation decouples router training from deployment-time budget selection, allowing a single trained router to support a wide range of operating points through threshold adjustment alone.

C. Threshold-based Decision

EENet [13], DynamicDet [2], RANet [12]

III. Method

A. Problem Formulation

AnyDepth-YOLO. We build on AnyDepth-YOLO [1], an object detector that enables multiple accuracy–efficiency operating points within a single model. Each stage in both the backbone and the detection neck consists of an essential path that always executes and a skippable refinement path, allowing the detector to operate at different execution depths. We consider two extreme configurations: a *super* path, which executes both the shared essential path and all refinement paths (highest accuracy, highest computation cost), and a *base* path, which executes only the shared essential path without any refinement (lowest computation cost with a modest accuracy drop).

Routing problem. We formulate the routing decision as a binary selection between a deep configuration δ_{super} and a shallow configuration δ_{base}. Although this yields discrete per-frame latencies, practical deployment constraints are typically imposed on average resource usage over time rather than on individual frames. Consequently, regulating the fraction of frames routed to δ_{super} enables continuous control of the detector's average operating point under varying resource budgets.

TABLE II
Router overhead (per frame, batch 1, input resolution 384×1248 for the KITTI dataset). Every variant adds negligible cost relative to the SUPER path (26.30 GFLOPs).

Router	Feature	#Params	GFLOPs	% of SUPER
GAP-MLP	backbone	60,241	1.17×10^{-4}	0.0004
TinyConv-MLP	backbone	60,881	4.17×10^{-4}	0.0015
	neck	52,433	3.51×10^{-4}	0.0013
	both	112,017	7.65×10^{-4}	0.0028

Specifically, for each input image, we define the *super advantage* as the reduction in detection loss obtained by using the deep path:

$$A = L_{\text{base}} - L_{\text{super}}, \tag{1}$$

where L_{base} and L_{super} are the per-image detection losses of the two paths. At deployment, the router exploits the strong temporal correlation between adjacent video frames, predicting an advantage score $\hat{A}$ from features generated on the previous frame and using a threshold τ to select the execution configuration for the current frame:

$$\text{config} = \begin{cases} \delta_{\text{super}}, & \hat{A} > \tau, \\ \delta_{\text{base}}, & \text{otherwise.} \end{cases} \tag{2}$$

B. Router Training

Figure 2 illustrates the router training pipeline. We formulate routing as a supervised regression problem, where the input consists of multi-scale intermediate features extracted from the frozen detector and the target is the corresponding super advantage A defined in Eq. 1. Intuitively, the router learns to predict how much an input would benefit from executing the deep path instead of the shallow path.

For each training image, we run both the SUPER and BASE execution paths once to obtain the ground-truth advantage together with the associated intermediate features. Since the detector remains completely frozen, the advantage serves as a stationary supervision signal throughout training. To improve efficiency, all features and targets are precomputed and cached offline, allowing the router to be trained without repeated detector forward passes.

Router Architecture.

The router predicts $\hat{A}$ from multi-scale features tapped from the frozen detector. Features can be extracted from the backbone, the detection neck, or their concatenation; we treat the feature source as a design choice and ablate it in Section IV-C1, adopting the concatenation of both by default. To keep routing overhead negligible, we explore two lightweight architectures. First, GAP-MLP discards spatial layout entirely by globally average-pooling each pyramid level and concatenating the resulting feature vectors. Second,

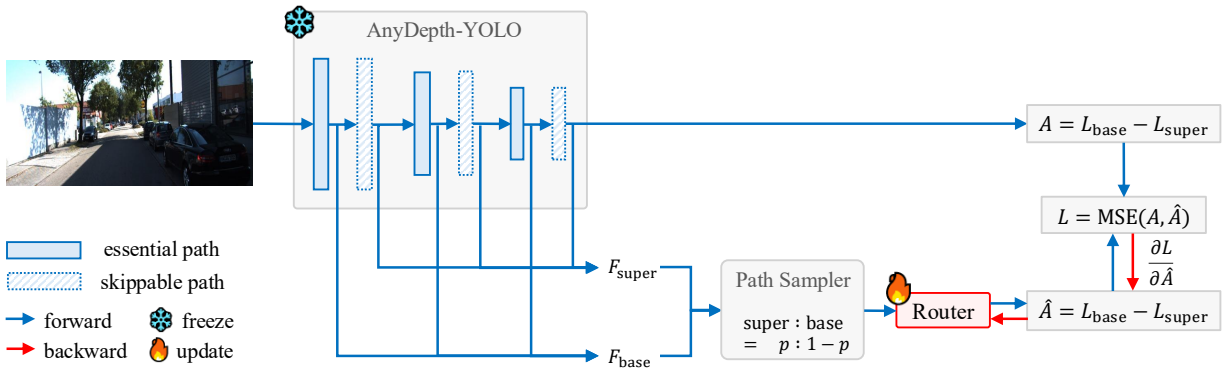


Fig. 2. Training pipeline. The AnyDepth-YOLO detector is frozen; both paths are run once per image to obtain the true advantage $A = L_{\text{base}} - L_{\text{super}}$, and only the router is updated. The router predicts $\hat{A}$ from the multi-scale intermediate features and is trained with $\text{MSE}(\hat{A}, A)$ (blue: forward, red: backward).

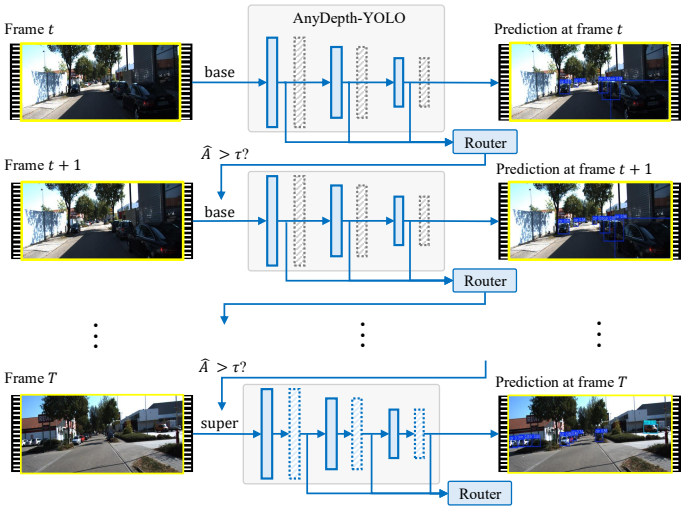


Fig. 3. Causal routing mechanism for video streams. At each frame the detector runs the chosen path and the router predicts $\hat{A}$ from its intermediate features; the decision $\hat{A} > \tau$ selects the path (base or super) for the next frame. Routing is causal and recursive across frames $t, t{+}1, \ldots, T$, adding only the lightweight router forward.

TinyConv-MLP preserves coarse spatial context: multi-scale features are first adaptively pooled to a common $G \times G$ grid (default 2×2) and stacked along the channel dimension, after which a lightweight 3×3 depthwise convolution performs spatial mixing before global pooling. In both variants, the final merged representation is directly mapped to $\hat{A}$ by a two-layer fully connected head. As shown in Table II, both designs introduce negligible computational overhead.

Training Objective. We train the router using a standard mean squared error (MSE) objective:

$$L_{\text{acc}} = \|A - \hat{A}\|^2. \tag{3}$$

Crucially, unlike efficiency-aware routing objectives of the form $\lambda_{\text{acc}} L_{\text{acc}} + \lambda_{\text{flops}} L_{\text{flops}} (+ \lambda_{\text{uni}} L_{\text{uni}})$, our objective contains no efficiency weight λ and no anti-collapse term. Consequently, the deployment operating point is not rigidly fixed during training, and the router simply learns to rank images based on how much the deep path improves detection.

Path-Conditioned Training. To ensure robust deployment, the router must remain well-calibrated to the feature distributions of both execution paths, as its input at test time depends entirely on whichever configuration was selected for the preceding frame. To prevent router training from favoring a single distribution, we simulate the prior frame's routing decision using a balanced action-sampling strategy. Specifically, for each training sample, we feed the router the features corresponding to the super path with probability p, and the base path features otherwise. This balanced exposure explicitly models the conditional execution loop of deployment, ensuring that the advantage predictor stays calibrated regardless of upstream routing choices.

C. Router Deployment

Once trained, the router operates as a causal, per-frame controller whose inference-time behavior is entirely governed by the routing threshold τ. To validate its practical utility, we evaluate the router under two distinct deployment paradigms based on how τ is managed: serving static operating points for standardized benchmarking, and dynamically tracking a latency budget for real-world application.

Threshold-Based Routing for Benchmarking. Because the router is trained solely to predict the expected loss reduction, the deployment operating point is fully decoupled from the training phase. The threshold τ in Equation (2) acts as a free dial: raising τ routes fewer images to the deep path (saving compute at the cost of accuracy), while lowering it does the opposite. Consequently, sweeping τ systematically shifts the dataset-averaged accuracy and computational cost, tracing a continuous accuracy–efficiency Pareto curve from a single trained model. For continuous video streams, routing is strictly causal (Figure 3). To prevent the router from introducing a latency bottleneck, the path decision for frame $t+1$ is made using the intermediate features already generated by the path executed on frame t. Exploiting the temporal continuity of video data, this allows the router to commit to a path at the start of each frame without requiring an extra detector forward pass.

Online Latency-Budget Control for Real-World Deployment. While a fixed τ maintains a static operating point,

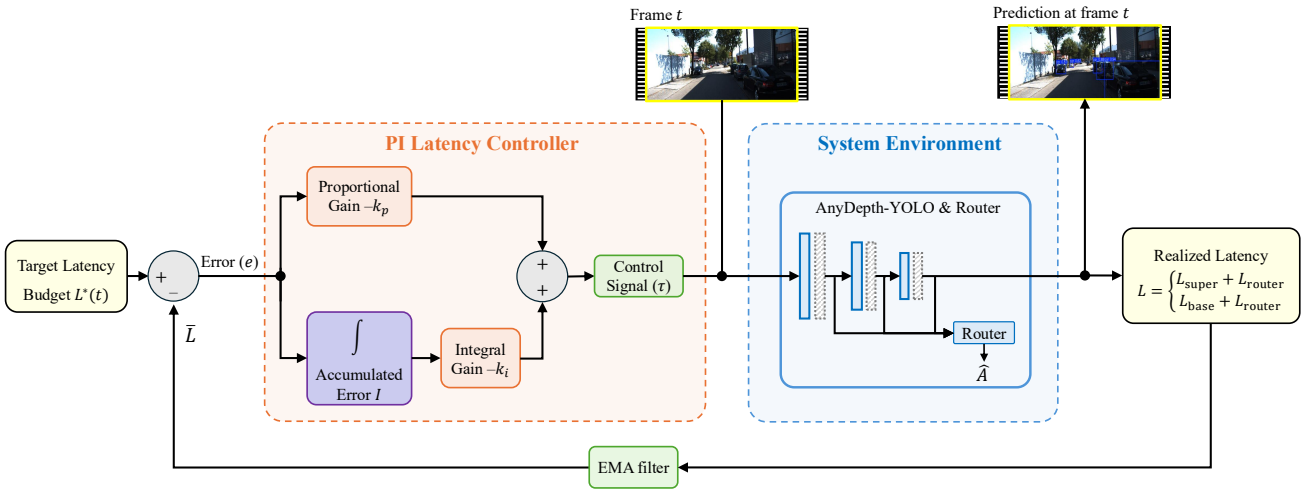


Fig. 4. PI control of the routing threshold τ. The error between the latency budget $L^\star(t)$ and an EMA $\bar{L}$ of the realized per-frame latency is fed through proportional (k_p) and integral (k_i) terms that nudge τ; a lower τ sends more frames to the super path (higher latency) and vice versa, closing the loop so the realized latency tracks the budget.

Algorithm 1 Causal budget-adaptive depth routing (deployment)

Input: frozen AnyDepth detector D, router R, target latency sequence $L^\star(t)$,
PI gains k_p, k_i, EMA momentum α, frames $x_1, \ldots, x_T$
Output: streaming per-frame detections y_t

$config \leftarrow \delta_{\text{base}}$ ▷ *initialize with base configuration*
$\tau \leftarrow \tau_0,\ \bar{L} \leftarrow L^\star(1),\ I \leftarrow 0$ ▷ *initialize PI controller state*
for $t = 1$ **to** T **do**:
  $(y_t, f_t) \leftarrow D(x_t, config)$ ▷ *execute chosen configuration*
  $\hat{A} \leftarrow R(f_t)$ ▷ *predict advantage for next frame*
  $config \leftarrow \delta_{\text{super}}$ **if** $\hat{A} > \tau$ **else** δ_{base} ▷ *route x_{t+1}*
  $l_t \leftarrow$ MeasureLatency() ▷ *observe realized per-frame latency*
  $\bar{L} \leftarrow (1-\alpha)\bar{L} + \alpha l_t$ ▷ *update EMA of latency*
  $e \leftarrow L^\star(t) - \bar{L}$ ▷ *compute latency tracking error*
  $I \leftarrow I + e$ ▷ *accumulate integral error*
  $\tau \leftarrow \tau - k_p e - k_i I$ ▷ *adjust threshold dynamically*
  emit y_t ▷ *stream detection output*
end for

real-world edge devices frequently face time-varying compute constraints (e.g., due to thermal throttling or battery limits). Thanks to the single-scalar nature of τ, the system can dynamically adapt to a shifting latency budget $L^\star(t)$ online without any retraining. As illustrated in Figure 4, we wrap the routing loop in a standard proportional–integral (PI) controller. The controller maintains an exponential moving average (EMA) $\bar{L}$ of the realized per-frame latency. It computes the error $e = L^\star(t) - \bar{L}$ and incrementally adjusts τ by $-k_p e - k_i I$ (where I is the accumulated integral error). Because lowering τ routes more frames to the super path (thereby increasing latency) and raising τ has the inverse effect, this closed-loop mechanism continuously nudges the realized latency to track the target budget. This threshold-based loop is summarized in Algorithm 1.

IV. Experiment

A. Experimental Setup

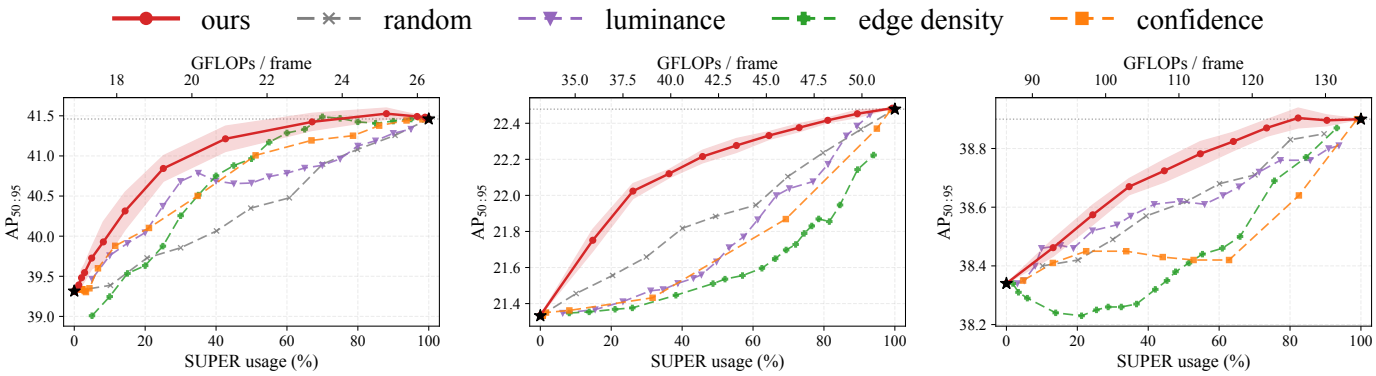
1) Datasets: We evaluate our approach on three representative driving benchmarks. For all datasets, the router is trained on static detection images and tested on continuous video sequences, without any temporal or tracking supervision at any stage. We assume no significant train-deployment distribution shift; the primary difference lies in the temporal structure of the deployment videos.

KITTI. We train the router on the KITTI Detection dataset [7] using an 80:20 split of 5,985 training and 1,496 validation images, and evaluate it on the KITTI Multi-Object Tracking benchmark consisting of 21 video sequences (8,008 frames). Frames are processed at a resolution of 384×1248.

BDD100K. To evaluate cross-dataset generalization, we train the router on BDD100K detection images [8] (70K train, 10K val) and evaluate on the BDD100K MOT validation split comprising 200 video sequences. Frames are processed at 720×1280 resolution.

Waymo. To further evaluate generalization on a large-scale, high-resolution benchmark, we use the front camera of the Waymo Open Perception dataset [9] (v2.0.1). As this benchmark provides no separate detection and tracking splits, we partition it by official split: the router is trained on the 798 training-split segments treated as static detection images (158.1K frames), and evaluated on the 202 validation-split segments as continuous video (39.1K labeled frames). Frames are processed at 1280×1920 resolution.

2) Baselines: To the best of our knowledge, no prior work has explored depth routing for AnyDepth-style detectors on continuous video streams. In the absence of established routing strategies, a natural starting point is to rely on human-interpretable notions of scene difficulty. One might reasonably expect deeper execution to be more beneficial for visually challenging inputs, such as poorly illuminated scenes, cluttered environments, or frames where the detector exhibits low confidence. Furthermore, many of these proxies



(a) KITTI (b) BDD100K (c) Waymo

Fig. 5. Pareto-front evaluation of depth routing on (a) KITTI-Tracking, (b) BDD100K MOT, and (c) Waymo Open. Each point on the proposed curve corresponds to a different routing threshold τ. We compare the advantage-regression router against random, luminance-, edge-density-, and confidence-based routing baselines in terms of AP@[.50:.95] versus SUPER-path usage. A single trained router traces a continuous accuracy–efficiency Pareto frontier that consistently dominates all baselines across all three benchmarks, with the largest gains observed in the low-compute regime where SUPER-path execution is limited.

TABLE III
EFFICIENCY OF THRESHOLD-BASED DEPTH ROUTING ACROSS THE OPERATING POINTS IN FIGURE 5. AP DENOTES AP@[.50:.95] (%). LATENCY, FPS, AND ENERGY ARE MEASURED ON AN RTX 3090 AND INCLUDE ROUTER OVERHEAD. ALWAYS-SUPER AND ALWAYS-BASE REPRESENT THE STATIC SINGLE-PATH BASELINES CORRESPONDING TO THE TWO ENDPOINTS OF THE PARETO FRONTIER.

(a) KITTI

τ	SUPER%	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)
ALWAYS-BASE	0	16.87	39.3	13.43	74.5	2099
+0.70	2.9	17.14	39.5	13.91	71.9	2133
+0.50	8.2	17.64	39.9	14.26	70.1	2196
+0.40	14.3	18.22	40.3	14.66	68.2	2268
+0.30	25.1	19.24	40.8	15.38	65.0	2396
+0.20	42.7	20.89	41.2	16.54	60.5	2604
+0.10	67.1	23.20	41.4	18.15	55.1	2893
+0.00	88.1	25.18	41.5	19.54	51.2	3142
−0.10	96.7	25.99	41.5	20.11	49.7	3244
−0.40	99.6	26.27	41.5	20.30	49.3	3278
ALWAYS-SUPER	100	26.30	41.5	20.03	49.9	3283

(b) BDD100K

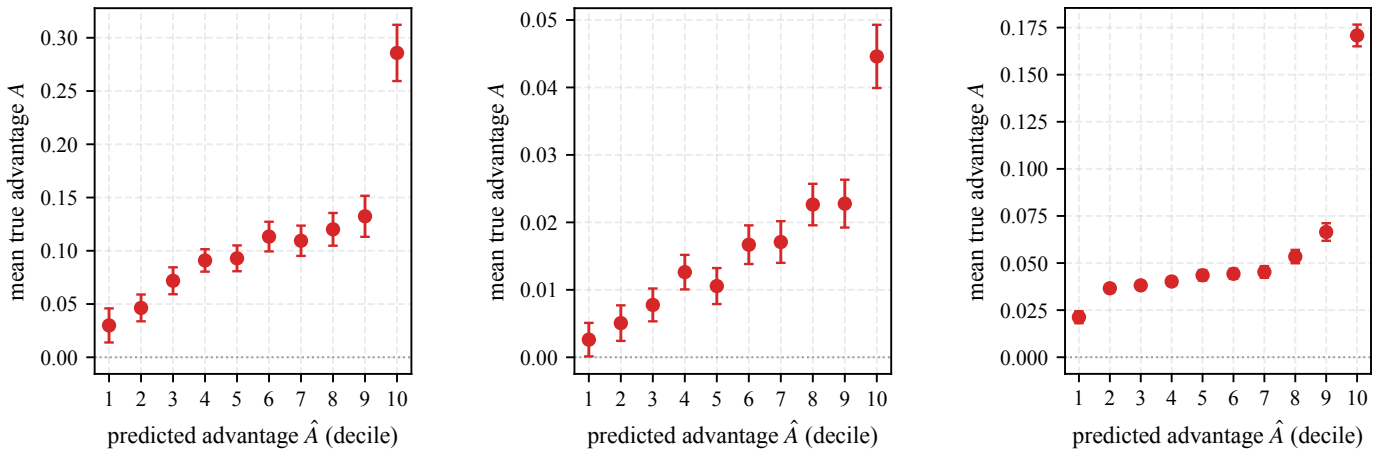
τ	SUPER%	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)
ALWAYS-BASE	0	33.18	21.3	25.41	39.4	8312
+0.086	14.8	35.92	21.8	27.86	35.9	9043
+0.072	26.1	38.02	22.0	29.52	33.9	9601
+0.064	36.3	39.91	22.1	31.01	32.2	10105
+0.058	45.8	41.66	22.2	32.41	30.9	10574
+0.054	55.3	43.43	22.3	33.80	29.6	11043
+0.049	64.5	45.14	22.3	35.15	28.5	11497
+0.045	73.0	46.72	22.4	36.39	27.5	11917
+0.040	81.1	48.21	22.4	37.58	26.6	12317
+0.033	89.4	49.76	22.5	38.79	25.8	12727
ALWAYS-SUPER	100	51.72	22.5	40.06	25.0	13250

(c) Waymo

τ	SUPER%	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)
ALWAYS-BASE	0	86.46	38.3	47.77	20.9	14606
+0.152	13.2	92.84	38.5	52.77	18.9	16248
+0.116	24.4	98.25	38.6	56.78	17.6	17640
+0.098	34.6	103.21	38.7	60.43	16.5	18909
+0.085	44.5	108.00	38.7	63.97	15.6	20140
+0.074	54.7	112.92	38.8	67.62	14.8	21408
+0.066	64.0	117.43	38.8	70.95	14.1	22564
+0.057	73.4	121.96	38.9	74.31	13.5	23733
+0.048	82.3	126.27	38.9	77.49	12.9	24840
+0.039	90.4	130.19	38.9	80.39	12.4	25847
ALWAYS-SUPER	100	134.84	38.9	83.54	12.0	27041

can be computed directly from simple image statistics without additional network inference or GPU computation, enabling extremely lightweight routing baselines. To benchmark our proposed advantage-regression router against these intuitive and lightweight cues, we establish the following heuristic baselines:

- **Random Routing:** Selects the execution path randomly based on a target probability, serving as a lower-bound baseline.
- **Luminance-based Routing:** Uses the mean pixel intensity of the input image as the routing signal, operating under the assumption that poorly lit scenes might require deeper execution.
- **Edge-density-based Routing:** Computes the proportion of edge pixels (e.g., via a Sobel operator) in the raw image, acting as a simple pixel-level proxy for scene clutter and object density.
- **Confidence-based Routing:** Uses the detector's own prediction confidence from the previous frame (specifically, the mean score of the top-20 bounding boxes) to decide the path for the current frame, assuming that low confidence indicates a difficult scene.



(a) KITTI (b) BDD100K (c) Waymo

Fig. 6. Router ranking reliability. Validation images are grouped into deciles based on the predicted advantage $\hat{A}$ (5-seed ensemble). Each point represents the mean true advantage $A = L_{\text{base}} - L_{\text{super}}$ within that decile (error bars denote standard error of the mean). The mean true advantage increases consistently with the predicted decile across all three datasets, demonstrating that a higher router score reliably identifies frames that benefit more from the deep path.

3) Implementation Details: The router is trained for 30 epochs (batch 256, Adam, lr 10^{-3}) over seeds 0–4 and reported as mean±std. For training efficiency, the router is trained on precomputed, frozen features without image-space augmentation. Given its minimal parameter count, regularization (e.g., weight decay, dropout) is omitted. Detection uses confidence threshold 0.25 and NMS IoU 0.7.

4) Evaluation Metrics: We report AP@[.50:.95], the average SUPER-path usage rate, and the average GFLOPs per frame over the evaluated video sequences. While detection accuracy (AP) is evaluated on an NVIDIA RTX 3090, all efficiency-related metrics, including latency, FPS, and energy consumption, are measured on an NVIDIA Jetson Orin Nano to validate the proposed budget-adaptive control framework under realistic edge deployment conditions. For this deployment-oriented evaluation, all models are converted to TensorRT [27] FP16 engines prior to benchmarking.

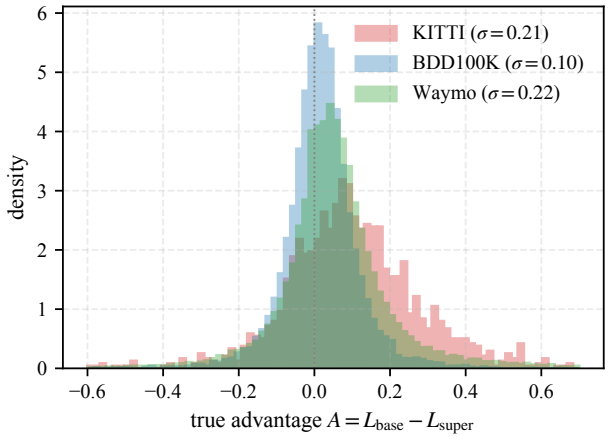


Fig. 7. Per-image advantage $A = L_{\text{base}} - L_{\text{super}}$ distribution on the validation sets. Both are concentrated near zero with a long positive tail—most frames gain little from the deep path while a minority gain a lot. The tail is wider on KITTI (σ=0.21) than on BDD100K (σ=0.10).

B. Results

1) Main Result: We compare the proposed advantage-regression router against the heuristic routing baselines. Figure 5 shows that a single trained router traces a continuous Pareto frontier that consistently dominates all baselines across all three benchmarks. In particular, the advantage of the proposed routing strategy is most pronounced in the low-budget regime, where selecting the right subset of frames for deeper execution is critical.

Table III provides the corresponding operating points along the Pareto curve. On KITTI (Figure 5(a)), the router maintains AP@[.50:.95] within 0.1 point of the SUPER-only ceiling (41.4 vs. 41.5) while routing only 67% of frames through the SUPER path. This reduces computation from 26.30 to 23.20 GFLOPs/frame (−11.8%), latency from 20.69 to 18.66 ms (−9.8%), and energy consumption from 3277 to 2957 mJ (−9.8%), while increasing throughput from 48.3 to 53.6 FPS (+11.0%).

The same trend holds on BDD100K (Figure 5(b)). At 55.3% SUPER usage, the router recovers nearly all of the SUPER-path accuracy gain (22.3 vs. 22.5 AP@[.50:.95]) while reducing computation from 51.72 to 43.43 GFLOPs/frame (−16.0%), latency from 40.06 to 33.80 ms (−15.6%), and energy consumption from 13250 to 11043 mJ (−16.7%). Throughput correspondingly increases from 25.0 to 29.6 FPS (+18.4%).

The same behavior carries over to the large-scale, high-resolution Waymo Open benchmark (Figure 5(c)). At only 35% SUPER usage, the router recovers nearly all of the SUPER-path accuracy gain (38.7 vs. 38.9 AP@[.50:.95], within 0.2 point of the ceiling) while reducing computation from 134.84 to 103.21 GFLOPs/frame (−23.5%), latency from 16.51 to 14.05 ms (−14.9%), and energy consumption from 2108 to 1677 mJ (−20.4%). Throughput correspondingly increases from 60.6 to 71.2 FPS (+17.5%). Notably, the advantage-regression router remains the only strategy that

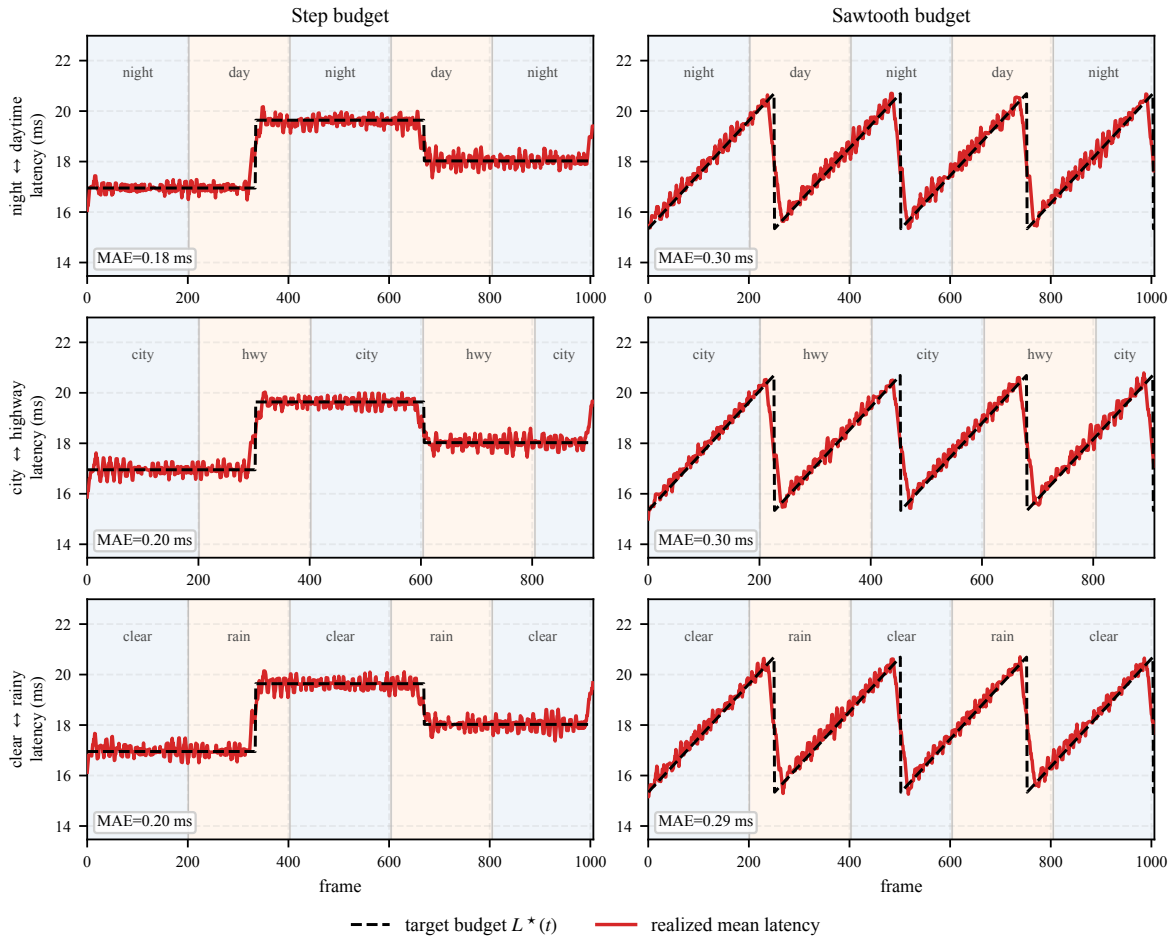


Fig. 8. Runtime latency-budget tracking under realistic scene-condition drift on BDD100K MOT, run as a live on-device closed loop. Sequences are concatenated in alternating condition order (night↔daytime, city↔highway, clear↔rainy; rows), with shaded bands marking each segment's condition. A PI controller adjusts the router threshold τ to hold a step (left) and a sawtooth (right) latency budget $L^\star(t)$ (black dashed); the realized mean latency (red) tracks it with MAE 0.18–0.30 ms across all condition drifts.

improves monotonically from the very first frames routed to the deep path, whereas the heuristic baselines (most visibly edge density) initially degrade accuracy below the always-base anchor by spending the SUPER budget on uninformative frames.

2) Advantage Regression Quality: Because deployment routes by thresholding the predicted advantage $\hat{A}$, the router need not predict the exact advantage magnitude; instead, it must correctly *rank* frames according to how much they benefit from the deep path. Figure 6 evaluates this property by grouping validation images into deciles based on $\hat{A}$ and reporting the mean true advantage $A = L_{\text{base}} - L_{\text{super}}$ within each group. On all three datasets, the mean true advantage increases consistently with the predicted decile, indicating that frames assigned higher scores indeed benefit more from deeper execution. This confirms that the router successfully captures the relative utility of the deep path, which is the key requirement for threshold-based routing. We emphasize ranking rather than per-image regression accuracy because the single-image target is inherently noisy, making robust ordering a more meaningful indicator of deployment performance.

The pronounced increase in the highest decile is a direct consequence of the true advantage's right-skewed distribution (Figure 7). While this rightward skew indicates that deeper execution generally yields lower detection losses, the distribution is heavily concentrated near zero. This means that for the vast majority of frames, the benefit of the super path is present but modest. In contrast, the long positive tail reveals that a small minority of frames experience substantial gains. The router successfully isolates these rare, high-payoff frames into the top-ranked decile, ensuring that limited computational resources are allocated precisely where deeper execution yields the greatest return. This positive tail is noticeably wider on KITTI (σ=0.21) and Waymo (σ=0.22) compared to BDD100K (σ=0.10). Accordingly, the advantage distribution on BDD100K spans a much narrower range, as reflected by the different y-axis scales in Figure 6, meaning that most achievable operating points for this dataset are governed by a relatively narrow band of threshold values τ.

3) Runtime Latency-Budget Tracking: The single-scalar threshold τ exposes the operating point as a tunable dial, allowing the detector to be steered at runtime to track a time-varying latency budget $L^\star(t)$ (e.g., a thermal- or battery-driven schedule) without any retraining. We wrap τ in a proportional-

integral (PI) control loop: the controller compares the target budget against an exponential moving average (EMA) of the realized per-frame latency and nudges τ to cancel the error. To stress the controller under realistic non-stationary conditions rather than abrupt synthetic boundaries, we concatenate BDD100K MOT sequences with alternating scene conditions (night↔daytime, city↔highway, clear↔rainy). This forces the input distribution—and hence the expected advantage of the deep path—to drift continuously as the scene evolves.

Because routing decisions are binary, each frame executes either the SUPER or BASE path, making fine-grained latency control impossible at the per-frame level. In practice, however, deployment constraints are typically imposed on average latency rather than individual frames. We therefore regulate the mean latency by adjusting the fraction of frames routed to the SUPER path through a PI controller driven by the EMA tracking error. Figure 8 evaluates this runtime tracking in a live on-device closed loop. To visualize the tracked mean without the phase delay introduced by causal trailing filters, we smooth the binary per-frame latency measurements using a zero-phase (centered) moving average. This offline visualization preserves the true tracking behavior while avoiding smoothing-induced lag, leaving the underlying control loop unchanged. The realized mean latency closely tracks both step and sawtooth target budgets across all condition drifts, achieving a mean absolute error of only 0.18–0.30,ms. Ultimately, a single trained router transforms a static Pareto front into a runtime-adjustable system, enabling precise latency-budget tracking despite relying on binary per-frame routing decisions.

4) TensorRT Deployment: A natural concern is whether depth routing remains practical on a production inference stack: TensorRT compiles a *static* graph and cannot conditionally skip layers inside a single engine. We therefore realize routing by exporting the BASE and SUPER paths as two separate FP16 engines and keeping both resident in GPU memory; the router simply selects which engine to execute per frame. The only routing-specific concern is then the cost of *switching* between the two resident engines every time the chosen path changes. We isolate this cost on an embedded NVIDIA Jetson Orin Nano by comparing each single-path engine run against an alternating schedule that forces a BASE↔SUPER engine switch on *every* frame—a pathological worst case far more frequent than any realistic routing trace, in which the chosen path typically persists for many consecutive frames. The switching overhead is then isolated as the latency of the alternating schedule minus the mean of the two static single-path runs. To verify that this holds across compute scales, we report it at two input resolutions—384×1248 and 736×1280 (the KITTI and BDD100K evaluation resolutions, respectively)—since it is the input resolution, not the dataset, that sets the per-engine cost (Table IV). Even under this per-frame worst case, switching costs only $+0.05\%$ at 384×1248 and $+0.16\%$ at 736×1280 over the single-path mean, confirming that engine switching is effectively free: because both FP16 engines are held resident in GPU memory, a switch merely selects which execution context to launch and incurs no weight reload or memory transfer.

Keeping all three engines (BASE, SUPER, and the router) co-resident is cheap on the embedded budget. At the higher 736×1280 resolution the full routed deployment occupies $214\,\text{MB}$ of GPU memory—$33\,\text{MB}$ of FP16 engine weights plus $181\,\text{MB}$ of activation scratch—of which the advantage-regression router is a negligible $1.3\,\text{MB}$; this is under 3% of the Orin Nano's $8\,\text{GB}$. The weights are resolution-independent (they are the detector parameters), whereas the activation scratch scales with input resolution, dropping to $97\,\text{MB}$ total ($130\,\text{MB}$ including weights) at 384×1248. The router head adds essentially nothing, since it consumes only the small grid-pooled feature taps the detector already produces.

TABLE IV
TENSORRT FP16 ENGINE-SWITCHING OVERHEAD ON THE EMBEDDED JETSON ORIN NANO (PURE GPU INFERENCE, CUDA-EVENT TIMED, 1000 ITERATIONS AFTER WARM-UP), REPORTED AT TWO INPUT RESOLUTIONS TO SPAN COMPUTE SCALES. ALTERNATING FORCES A BASE↔SUPER ENGINE SWITCH ON EVERY FRAME (PER-FRAME WORST CASE); AT BOTH RESOLUTIONS ITS LATENCY STAYS WITHIN 0.2% OF THE MEAN OF THE TWO STATIC SINGLE-PATH ENGINES, SO ROUTING BETWEEN TWO RESIDENT ENGINES IS EFFECTIVELY FREE. RESIDENT GPU MEMORY (FP16 WEIGHTS + ACTIVATION SCRATCH) IS REPORTED PER ENGINE; THE ROUTER IS NEGLIGIBLE.

Engine schedule	Lat. (ms)	FPS	Energy (mJ)	Mem (MB)
(a) 384×1248 *(KITTI resolution)*				
BASE (always-base)	12.74	78.5	218.7	61.7
SUPER (always-super)	21.10	47.4	381.4	68.3
ALTERNATING (switch/frame)	16.93	59.1	300.4	—
mean(base, super)	16.92	—	—	**130.0**
switch overhead	$+0.009$ ms	$(+0.05\%)$		
(b) 736×1280 *(BDD100K resolution)*				
BASE (always-base)	24.88	40.2	485.1	102.8
SUPER (always-super)	41.67	24.0	842.6	110.0
ROUTER		(advantage head)		1.3
ALTERNATING (switch/frame)	33.33	30.0	666.9	—
mean(base, super)	33.27	—	—	**214.1**
switch overhead	$+0.052$ ms	$(+0.16\%)$		

One deployment caveat concerns the operating threshold itself. FP16 quantization slightly perturbs the router's predicted advantage $\hat{A}$, so a threshold τ calibrated in FP32 realizes a different SUPER usage once the engines run in FP16 (e.g., on BDD100K, τ=0.086 routes 14.8% of frames to the deep path in FP32 but 5.9% in FP16). Crucially, quantization shifts only the *calibration* of τ, not the ranking it induces: $\hat{A}$ remains monotone, so the same accuracy–efficiency frontier is reachable, merely at different threshold values. Recovering a target operating point therefore requires no retraining—only a one-pass recalibration of the τ-to-budget map on a small held-out clip evaluated through the FP16 engines. In the budget-tracking deployment (Sec. IV-B3) even this step is unnecessary: the PI controller adjusts τ online from the realized latency, absorbing the FP16 shift automatically.

C. Ablation Studies

1) Input Feature: Backbone vs. Neck vs. Both: We evaluate three potential feature sources for the router: the backbone pyramid features (input-level representations extracted before the detection neck), the neck features (prediction-level representations immediately preceding the detection heads), and

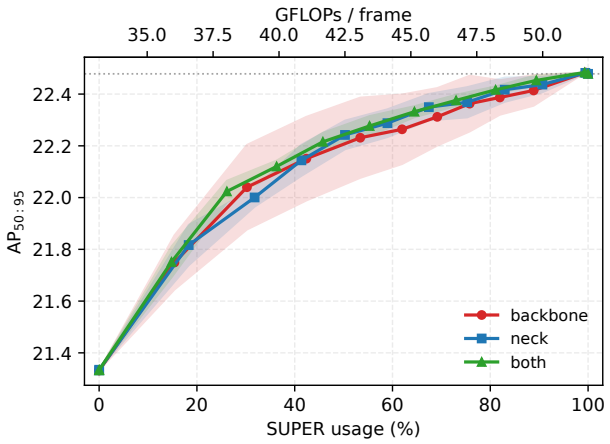


Fig. 9. Ablation on feature sources for the router on BDD100K. Using both backbone and neck features yields the best-or-tied mean performance and the most stable routing behavior across training seeds.

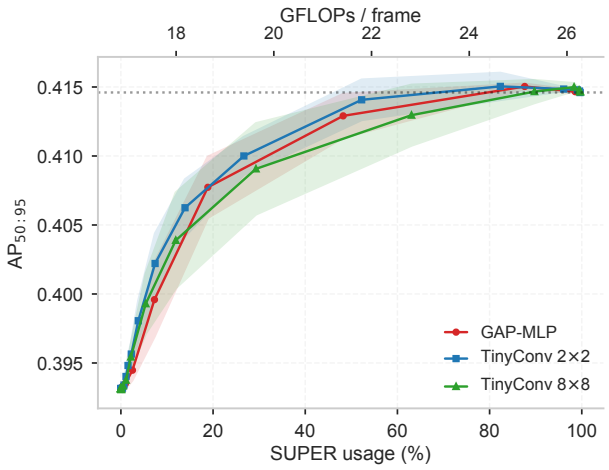


Fig. 10. Ablation on router architecture and grid resolution on KITTI.

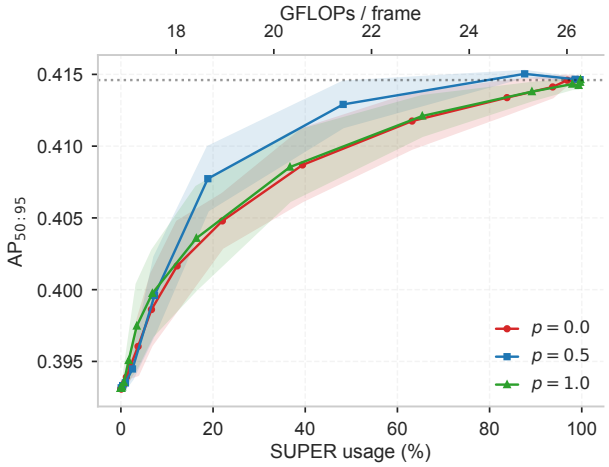


Fig. 11. Ablation on the previous-action sampling ratio p on KITTI. The balanced setting ($p = 0.5$) achieves the most robust performance by reducing train-deployment distribution shift.

their concatenation (both). Figure 9 presents the results on BDD100K.

While all three configurations exhibit similar mean accuracy–efficiency trade-offs, their stability differs noticeably across five training seeds. The backbone-only router shows the widest performance variance (red shaded region). Because backbone features represent the visual encoding prior to multi-scale aggregation, they lack the fully fused, task-aligned context that directly dictates the final prediction difficulty. Relying solely on these pre-fusion features forces the router to implicitly infer the outcome of the complex neck operations, making the routing policy more sensitive to initialization and training noise.

In contrast, combining the backbone and neck features consistently achieves the best-or-tied mean performance while exhibiting the narrowest variance band. By providing the router with both the raw hierarchical feature state and the aggregated, task-specific context, the network observes a more comprehensive representation of the frame's inherent difficulty. This enables a much more robust and consistent estimation of the deep-path advantage. Since this combined design incurs negligible additional computational overhead (Table II), we adopt the *both* configuration as the default feature source throughout the paper.

2) Router Design and Grid Resolution: We evaluate the impact of spatial resolution on the router's predictive capacity by comparing a spatially-agnostic GAP-MLP with TinyConv architectures utilizing 2×2 and 8×8 pooling grids (Figure 10). As illustrated in the performance trajectories, the TinyConv router with a coarse 2×2 grid consistently achieves the optimal accuracy–efficiency trade-off. The performance gap between the 2×2 variant and GAP-MLP highlights the necessity of spatial awareness. Because GAP-MLP globally collapses the feature maps, it entirely discards spatial layout. The superiority of the 2×2 grid demonstrates that preserving a coarse geometric context, which enables the router to roughly localize dense or difficult object clusters within the frame, provides critical cues for estimating the utility of the deep path. Conversely, increasing the spatial grid resolution to 8×8 proves detrimental. This high-resolution configuration not only yields the lowest mean performance but also suffers from severe instability across different training seeds, as evidenced by its significantly wider variance band (green shaded area). This instability suggests that forcing a lightweight router to process overly fine-grained spatial details causes it to overfit to local feature noise rather than learning a robust, generalized representation of overall scene difficulty. Consequently, the 2×2 grid emerges as the optimal configuration, supplying sufficient spatial context to effectively inform routing decisions while avoiding the optimization challenges associated with higher resolutions.

3) Previous-Action Sampling Ratio $p(\text{super})$: During training, we simulate the prior frame's routing decision by sampling the assumed previous action as super with probability p, where p represents the path whose features the router conditions on. We evaluate three configurations: $p \in \{0.0, 0.5, 1.0\}$. As illustrated Figure 11, the balanced setting of $p = 0.5$ yields the most robust performance, consistently outperforming both extreme configurations. This behavior is driven by the need to mitigate train-deployment distribution shift. At deployment,

the router must process features generated by whichever path actually executed on the previous frame. Since the two paths produce slightly different feature representations for the same input, training on only one path ($p = 1.0$ or $p = 0.0$) leaves the router poorly calibrated when the other path is encountered at runtime. In contrast, $p = 0.5$ exposes the router equally to both feature distributions, resulting in more robust advantage estimation and more stable routing decisions during causal video inference.

V. Conclusion

We reformulated dynamic-depth routing as advantage regression, decoupling the training objective from the deployment operating point. A single lightweight router predicts the per-image super advantage from the detector's intermediate features, and a threshold sweep recovers an entire continuous accuracy–efficiency Pareto curve with no trade-off weight and no anti-collapse term. On KITTI-tracking the router Pareto-dominates random, luminance, edge-density, and confidence baselines and comes within 0.1 AP of super-only accuracy at a 12% FLOPs (and 10% energy) reduction, at negligible overhead. A limitation is that the regression view targets the two-path (super/base) decision; extending it to multi-level or per-layer (2^N) routing, where the decision space is no longer enumerable and sampling- or marginalization-based estimators become necessary, is left to future work. A further limitation is that, with a binary per-frame actuator, what we regulate is the *average* latency (throughput/energy/thermal budgets); hard per-frame real-time deadlines are guaranteed only when the super path itself meets the deadline, and tighter per-frame guarantees would require finer-grained multi-level depths, also left to future work.

References

[1] W. Kang, H. Lee, and J. Lee, "Anydepth-detr/-yolo: Any-depth object detection with a single network," *arXiv preprint arXiv:2605.09407*, 2026.

[2] Z. Lin, Y. Wang, J. Zhang, and X. Chu, "Dynamicdet: A unified dynamic architecture for object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 6282–6291.

[3] D. Kuhse, H. Teper, S. Buschjäger *et al.*, "You only look once at anytime (AnytimeYOLO): Analysis and optimization of early-exits for object-detection," *arXiv preprint arXiv:2503.17497*, 2025.

[4] Y. Guo, H. Shi, A. Kumar, K. Grauman, T. Rosing, and R. Feris, "Spottune: transfer learning through adaptive fine-tuning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 4805–4814.

[5] Y. Meng, C.-C. Lin, R. Panda, P. Sattigeri, L. Karlinsky, A. Oliva, K. Saenko, and R. Feris, "Ar-net: Adaptive frame resolution for efficient action recognition," in *European conference on computer vision*. Springer, 2020, pp. 86–104.

[6] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "Blockdrop: Dynamic inference paths in residual networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8817–8826.

[7] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.

[8] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multitask learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2636–2645.

[9] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2446–2454.

[10] S. Teerapittayanon, B. McDanel, and H. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. Int. Conf. Pattern Recognit. (ICPR)*, 2016, pp. 2464–2469.

[11] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=Hk2aImxAb

[12] L. Yang, Y. Han, X. Chen, S. Song, J. Dai, and G. Huang, "Resolution adaptive networks for efficient inference," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2369–2378.

[13] F. Ilhan, K.-H. Chow, S. Hu, T. Huang, S. Tekin, W. Wei, Y. Wu, M. Lee, R. Kompella, H. Latapie *et al.*, "Adaptive deep neural network inference optimization with eenet," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 1373–1382.

[14] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. Vetrov, and R. Salakhutdinov, "Spatially adaptive computation time for residual networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1039–1048.

[15] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang, "Slimmable neural networks," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=H1gMCsAqY7

[16] J. Yu and T. S. Huang, "Universally slimmable networks and improved training techniques," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 1803–1811.

[17] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once for all: Train one network and specialize it for efficient deployment," in *International Conference on Learning Representations*, 2020.

[18] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "Skipnet: Learning dynamic routing in convolutional networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 409–424.

[19] W. Kang and H. Lee, "Adaptive depth networks with skippable sub-paths," *Advances in Neural Information Processing Systems*, vol. 37, pp. 33 213–33 231, 2024.

[20] L. Yang, Z. Zheng, J. Wang, S. Song, G. Huang, and F. Li, "AdaDet: An adaptive object detection system based on early-exit neural networks," *IEEE Trans. Cogn. Develop. Syst.*, vol. 16, no. 1, pp. 332–345, 2024.

[21] S. Heo, S. Cho, Y. Kim, and H. Kim, "Real-time object detection system with multi-path neural networks," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2020, pp. 174–187.

[22] A. Soyyigit, S. Yao, and H. Yun, "VALO: A versatile anytime framework for LiDAR-based object detection deep neural networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 4045–4056, 2024.

[23] X. Dai, D. Chen, M. Liu, Y. Chen, and L. Yuan, "Da-nas: Data adapted pruning for efficient neural architecture search," in *European conference on computer vision*. Springer, 2020, pp. 584–600.

[24] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=B1ckMDqlg

[25] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[26] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby, "Scaling vision with sparse mixture of experts," *Advances in Neural Information Processing Systems*, vol. 34, pp. 8583–8595, 2021.

[27] NVIDIA, "NVIDIA TensorRT," https://developer.nvidia.com/tensorrt, 2025. [Online]. Available: https://developer.nvidia.com/tensorrt